

RISE - Interview questions - and sample answers

40 questions | Java application logic | Rise 2.1

A practice guide based on the delivered source code and verification records. Use the answers as a starting point, then explain each idea in your own words.

Your 45-second introduction

Rise is a Java-based offline Android fitness and productivity app. I focused on task management, local persistence, recurring alarm schedules and gamified progress. A task has selected days and a time; completion is stored by task and date, which prevents duplicate XP. The app also supports strict daily streaks, reusable plans and timed rewards. I verified the logic with Java checks and Android instrumentation tests. It currently has no remote backend; adding a Spring Boot service would be a future extension.

How to practice

1. Read one topic and answer aloud without reading the sample. 2. Open the code anchor and trace a real example. 3. Practice the limitation or follow-up below each answer. 4. Explain what is implemented, what is tested and what you would improve.

Accuracy reminder

Current implementation: Java, native Android, SharedPreferences, JSON, AlarmManager and local business rules. Future proposals: Spring Boot APIs, a relational server database, login and cloud synchronization. Do not present these proposals as completed features.

Prepared from the project files and Rise 2.1 verification record. Counts and implementation details describe this release, not guaranteed behavior on every device.

Explain the project

01. Tell me about Rise.

Rise is an offline Android fitness and productivity tracker built in Java. It combines daily tasks, home workouts, meal guidance and reminders with XP, ranks and streaks. My interview focus is the application logic: task CRUD, recurrence calculations, local storage, backup validation, progression rules and timed events. The delivered app is a native APK, not a website wrapper.

Follow-up / evidence: Start with the problem, then name two technical components. Anchor: Task.java, Store.java and Progress.java.

02. What problem does the project solve?

It brings a daily routine into one place: workouts, care, coding practice and job preparation. Users can edit tasks, choose reminder days and times, hide completed tasks and reuse plans. The game-style progress gives feedback on consistency. I would describe it as a personal habit tool; I would not claim measured health improvements or user retention without collecting evidence.

Follow-up / evidence: Distinguish implemented functionality from unmeasured business outcomes.

03. Is this a Java backend project?

It demonstrates Java business logic and persistence inside an Android application. It does not currently have a remote backend, Spring Boot service, REST endpoints or MySQL database. For a backend interview, I explain the transferable concepts: domain modeling, validation, idempotent completion state, scheduling, serialization and testing. I keep the Android implementation separate from a proposed server architecture.

Follow-up / evidence: Say "Java application/domain logic" rather than claiming an implemented server backend.

04. What functionality did you focus on?

The project work covers task creation and updates, weekday scheduling, completion records, local JSON persistence, strict streak ranks, reusable plans and bonus-event rewards. I also worked through APK packaging and verification. When discussing my contribution, I can trace a requirement to the relevant class and its tests, and I disclose AI assistance if asked instead of claiming every line was written unaided.

Follow-up / evidence: Prepare to open one real method and explain its input, decisions and output.

Architecture and Core Java

05. How is the application organized?

Task models a scheduled activity. Store reads and writes local state. Progress derives XP, levels, ranks and statistics from completion records. Reminders and the broadcast receivers manage scheduled notifications. Challenges defines daily activities and event timing. Plans manages routine templates. MainActivity and the UI helpers connect these parts to native Android views. The separation is practical, but it is not a full clean-architecture implementation.

Follow-up / evidence: Improvement: introduce repository interfaces and move more business decisions out of UI classes.

06. Where did you apply object-oriented programming?

I used classes to group related state and behavior: a Task contains its schedule and can determine whether it runs on a date; Store handles persistence; Progress calculates derived results. BroadcastReceiver subclasses implement Android callbacks. I favor composition between these components. Encapsulation could improve because several fields and helpers are exposed directly rather than protected behind a small interface.

Follow-up / evidence: Do not force claims about every OOP principle. Give one concrete class example.

07. Which Java collections did you use, and why?

ArrayList stores task collections that can be sorted by time. HashMap maps task IDs to categories for lookup while calculating statistics. TreeMap keeps daily counts and XP awards in date order. HashSet stores cleared dates so the streak calculation can check membership efficiently. These choices match list traversal, lookup, chronological reporting and uniqueness requirements.

Follow-up / evidence: HashMap/HashSet lookup is expected constant time; TreeMap operations are logarithmic in its entry count.

08. What are the main complexity and scalability concerns?

Task scheduling scans a fixed small number of dates, while loading and sorting tasks grows with the number of tasks. Progress scans preference records, aggregates dates and also checks historical completion dates against task definitions. That history reconstruction can approach $O(D \text{ times } T)$, where D is the number of historical dates and T is the task count. It is not just one constant-time lookup.

Follow-up / evidence: For larger histories, use indexed storage, stable daily summaries and background computation.

Persistence, CRUD and validation

09. How do you store data?

Store uses Android SharedPreferences. Task lists and other structured collections are serialized as JSON strings; completion records are dated boolean keys. This kept the personal offline application simple and dependency-light. It is suitable for modest local data, but it lacks relational queries, indexes and database transactions across the entire workflow. A larger version should move to Room or another structured persistence layer.

Follow-up / evidence: Actual storage: SharedPreferences and JSON. No SQLite/Room or MySQL is implemented.

10. Walk me through creating or editing a task.

The editor validates the title, duration, time and repeat selection. A new task receives a persisted nextId value; an edit retains its existing ID. The app cancels the old reminder, saves the updated task list and schedules the next occurrence. It then refreshes the UI. IDs are generated locally with a read-modify-write approach, which is not a distributed or generally thread-safe ID service.

Follow-up / evidence: Discuss cancellation before rescheduling so an old time does not leave a stale alarm.

11. How does task completion avoid duplicate rewards?

Completion is stored as a boolean for a task ID and local date, such as done_12_2026-09-24. XP is derived from true completion records rather than incremented every time a button is tapped. Repeating the same completion writes the same state, so it does not add another reward. Unchecking the task changes the boolean to false and removes that completion from the derived XP total.

Follow-up / evidence: This is local idempotent state, not a claim of distributed exactly-once processing.

12. How do backup and restore work?

The application exports local state to JSON. Before restore, it validates the backup format, task fields, IDs, schedule ranges, profile values and supported plan/event fields, then rebuilds an allowed set of values. It cancels existing task alarms, restores local state and reschedules. The export is a user-managed file, not an encrypted cloud backup. Restore could be strengthened with atomic staging and rollback.

Follow-up / evidence: Never equate input validation with encryption, authentication or a transactional restore guarantee.

Dates, recurrence and alarms

13. How are selected weekdays represented?

`Task.days` uses a seven-bit mask: Monday is bit 0 and Sunday is bit 6. To check a date, the code computes one shifted bit and applies a bitwise AND. All seven days is 127; Monday, Wednesday and Friday is $1 + 4 + 16$, or 21. A zero mask represents a one-time task whose separate date field is checked.

Follow-up / evidence: Code: `(days & (1 << (date.getDayOfWeek().getValue() - 1))) != 0`.

14. How do you calculate the next reminder?

`Task.next` converts the current instant into the configured `ZonedDateTime`, then examines today and the following eight dates. For each selected date it combines the stored hour/minute with that date and timezone and requires the resulting instant to be strictly in the future. A one-time task beyond the scan window is handled separately. If nothing valid remains, it returns -1.

Follow-up / evidence: The scan is bounded; the actual implementation uses nine candidate dates, not an unlimited loop.

15. How do you handle timezones and midnight?

The application uses `java.time` types such as `LocalDate`, `Instant`, `ZonedDateTime` and `ZonedDateTime`. Completion keys use the user-selected local date, while alarms use epoch milliseconds. The daily reset is the next local midnight, rather than now plus a fixed 24 hours. Java timezone rules resolve date/time conversion. Changing the configured timezone can change which local day a user is currently in.

Follow-up / evidence: Tests cover date boundaries and timezone/DST cases; this is not proof of every possible clock-change scenario.

16. Are alarms guaranteed to fire exactly on time?

No. With exact-alarm access, recurring task occurrences use `AlarmManager.setAlarmClock`. Other timed reminders use exact scheduling when allowed; an approximate fallback is available when permission is missing. Notifications need their own permission and channel settings. Device restrictions, force-stop and volume settings still matter. The app posts an alarm-sound notification; it is not a continuously ringing full-screen clock alarm.

Follow-up / evidence: Be precise about exact-alarm access versus notification permission.

Receivers, failure handling and lifecycle

17. What happens when an alarm fires?

ReminderReceiver reads the task and event date. For a normal FIRE event it schedules the next occurrence first. It then checks whether the event belongs to today and whether the task was already completed. Eligible tasks produce a notification with Done and Snooze actions. Done writes completion and clears follow-ups. Snooze schedules another reminder ten minutes later.

Follow-up / evidence: Anchor: ReminderReceiver.onReceive. A completed task can remain scheduled internally but produce no alert for that date.

18. How do you avoid duplicate or stale alarms?

PendingIntent identity combines the task ID, an action and a request-code offset for different reminder kinds. The scheduler cancels the existing normal alarm before registering its next occurrence. Editing or deleting a task cancels its reminders, including follow-ups. PendingIntents are immutable and update their stored extras. This manages local alarm identity; it does not make every surrounding storage operation transactional.

Follow-up / evidence: Differentiate normal FIRE, snooze and nudge actions.

19. What happens after a reboot or app restart?

BootReceiver clears temporary snooze/nudge reminders and rebuilds future routine alarms from persisted tasks. The activity also refreshes scheduling when resumed. Saved task and completion data survive normal app restarts. Force-stop is a separate platform state: the user needs to reopen the app to restore its future scheduling behavior. I would test reboot restoration and process recreation explicitly.

Follow-up / evidence: Source: BootReceiver.java and the activity lifecycle callbacks.

20. How do you handle errors and invalid input?

User input is checked before saving, and malformed backup data is rejected before restore begins. The scheduler handles missing exact-alarm access with a fallback. The code catches failures in file, JSON and notification operations. Some low-level catches are intentionally broad or silent, which is a limitation: a stronger version should distinguish validation errors from unexpected failures and record useful diagnostics without exposing private data.

Follow-up / evidence: Acknowledge broad exception handling as a refactoring opportunity, not best practice.

XP, streaks and rank logic

21. Explain the XP and level calculation.

A normal dated task completion contributes 50 XP. Each of two optional daily bonus quests contributes 150 XP, and a completed timed event contributes 250 XP. Lifetime XP is the sum of eligible recorded rewards. Level is 1 plus integer division of lifetime XP by 500. Bonus rewards affect XP, but they do not substitute for clearing the scheduled routine to maintain the strict streak.

Follow-up / evidence: Example: 750 lifetime XP gives level 2, with 250 XP toward the next level.

22. What counts as a streak day in version 2.1?

Every task scheduled for that local date must be completed, and at least one task must be scheduled. Store records a cleared-date snapshot. Undoing a task or changing today's routine recalculates today's status. While today is unfinished, a streak ending yesterday remains active. Missing an entire day breaks it. Rest days should contain a light recovery or care task if the user wants streak credit.

Follow-up / evidence: Earlier versions counted any completion; the current release requires the full scheduled day.

23. How does promotion and reset work?

Rank depends on both consecutive cleared days and XP earned during that current streak. For example, D requires one day and 100 XP, S requires seven days and 1,500 XP, and SSSS requires 21 days and 5,000 XP. A broken streak resets rank to E and begins a new rank-XP run. Lifetime XP, level and workout history are retained.

Follow-up / evidence: The app keeps lifetime progression and current-run rank as separate concepts.

24. What happens when tasks or historical routines change?

Deleting a task definition keeps its dated completion records and earned XP. Its category-based points fall back to the personal category if the definition is missing. Recorded historical clear-day snapshots stay stable. Older records without snapshots are reconstructed using the current task definitions, so they have a historical-accuracy limitation. A stronger model would store the required task IDs and schedule version for each day.

Follow-up / evidence: Important tradeoff: an editable current schedule is not an immutable historical schedule.

Random quests, timed events and plans

25. Are the daily bonus quests truly random?

The app makes a deterministic daily draw. It copies a fixed activity list and shuffles it using a seed based on the local date, then selects two distinct items. Reopening the app that day gives the same draw, preventing simple rerolls. It is not cryptographic randomness, and users on the same date can receive the same selection. The timed-event activity also rotates by date.

Follow-up / evidence: Source: Challenges.daily and Challenges.event.

26. How does a quick-time event work?

The event start timestamp is persisted. A claim is valid only after that start and before the earlier of start plus five minutes or the next local midnight. A daily claimed flag prevents another reward. Closing the screen does not pause the deadline. After expiry the same date cannot restart the event. Completion is self-reported; there is no camera verification or server authority.

Follow-up / evidence: At the exact deadline the event is expired because the comparison is strictly less than the deadline.

27. Can users cheat by changing the phone clock?

This is an offline personal tool, not a competitive reward platform. The event check rejects a current time earlier than its saved start, but wall-clock-based timing and editable local data are not tamper-proof. A trusted server timestamp, authenticated event records and server-side claim validation would be needed for stronger enforcement. I would not claim complete anti-cheat protection for this version.

Follow-up / evidence: Foreground duration measurement could also use a monotonic clock, while handling reboot boundaries separately.

28. How do preset and personal plans work?

Four templates generate editable Task objects: Balanced Day, Fitness and Recovery, Career Builder and Gentle Restart. Users preview days and times, then add or replace their routine. Adding skips matching title/time/day combinations; replacement cancels old alarms but keeps history. Personal plans store a name and task templates as JSON. Applying a template assigns new task IDs, and one-time entries move to today.

Follow-up / evidence: Matching title/time/days is practical duplicate detection, not a complete semantic equality check.

Timers, resources and local safety

29. Why does the workout stopwatch survive leaving the screen?

The app stores a start timestamp and accumulated elapsed duration. Displayed elapsed time is the saved duration plus the difference between now and start while running. Pause saves that total and clears the active start. The UI handler refreshes the display; it is not the source of elapsed time. Finishing writes a session log and resets the timer state.

Follow-up / evidence: Rest-timer sounds are foreground-only. Do not confuse them with persistent task alarms.

30. What prevents image loading from consuming excessive memory?

The image loader reads bitmap dimensions first and selects a sampling factor before decoding the full image. It also uses an LruCache sized by bitmap byte count, capped at 12 MiB. This reduces repeated decoding and the size of cached images. Views can still retain displayed bitmaps, so the cache limit is not a guarantee that the entire app uses only 12 MiB.

Follow-up / evidence: Source: MainActivity.loadPicture. Image files and the PDF are bundled locally.

31. What privacy and security features are actually present?

The app stores its data locally, has no login, analytics or Internet permission, and does not embed the supplied personal photographs. YouTube links open in an external application. App-private storage reduces casual access, but the JSON export is a readable user-controlled file. This is not encrypted cloud storage, and there is no authentication or multi-user authorization layer.

Follow-up / evidence: Describe actual boundaries, not a blanket claim that local data is completely secure.

32. Is SharedPreferences safe for every workflow in this project?

It is convenient for small settings and lightweight local state, but the current implementation stores growing JSON collections and performs synchronous commits in several flows. Read-modify-write operations such as local ID allocation are not safe as a general concurrent service design. For larger data or concurrent writers, I would use a transactional repository, background I/O and explicit uniqueness constraints.

Follow-up / evidence: Differentiate a preference write from an atomic business transaction spanning multiple operations.

Testing and a real debugging example

33. How did you test Rise?

The verified 2.1 release passed 266 scheduling/serialization checks and 536 progression/event checks in Java, for 802 automated Java checks. It also passed 69 core Android instrumentation checks and 51 new plan/streak/event checks on an Android 15 emulator. Asset checks inspected the APK, PDF and calendar. I separate these checks from physical-device and real-world user testing, which were not performed.

Follow-up / evidence: These counts are assertions/checks, not 802 JUnit test methods or a code-coverage percentage.

34. Which edge cases were most important?

I focused on weekday-mask combinations, one-time dates, midnight and timezone boundaries, repeated completion, undo, partial versus fully cleared days, missed-day resets, every rank threshold, future/malformed records and timed-event expiry. Device tests covered preset duplication, replacement, saved plans, actual alarm delivery and next occurrence scheduling. Those cases exercise business rules that a successful happy-path UI demo would miss.

Follow-up / evidence: Be ready to describe one input and its exact expected result.

35. Describe a difficult bug and how you diagnosed it.

One APK contained exercise images that passed ZIP-level checks, but Android could not open them through AssetManager. Device diagnostics exposed Windows path separators in the archive entry headers. The build pipeline was changed to normalize ZIP entry names before alignment and signing. A regression check now compares local and central ZIP header paths, and device tests decode the images.

Follow-up / evidence: Evidence: NormalizeApk.java and tests/check_artifacts.py. The key lesson is to test the installed artifact.

36. What was the final verification limit?

The app was installed and tested on an Android 15 emulator, while Android 8 is the declared minimum supported version. That does not prove behavior on every Android version or manufacturer. Audible output on the user's physical phone and every battery restriction were not verified. I would next test a small physical-device matrix, reboot recovery, permissions denied, process death and long-running use.

Follow-up / evidence: Avoid saying "works perfectly on all Android phones".

Backend design follow-ups - proposed work

37. How would you add a Spring Boot backend?

I would first define API contracts and a domain model for users, task definitions, scheduled occurrences, completions and reward claims. Controllers would validate requests, services would enforce rules, and repositories would manage persistence. Authentication would identify the owner of each resource. The Android app would become a client with an offline queue. This is a proposed extension, not part of the delivered APK.

Follow-up / evidence: Possible endpoints: POST /tasks, PATCH /tasks/{id}, PUT /tasks/{id}/completions/{date}.

38. What database design would you propose?

I would use separate tables for users, tasks, schedule versions, completions, daily clear snapshots, plans and reward claims. A completion could have a unique key on user ID, task ID and local date. A reward claim could be unique by user ID, reward type, date and slot. Historical schedule snapshots would make streak calculation independent of later task edits.

Follow-up / evidence: Proposed schema only. Current storage remains SharedPreferences/JSON.

39. How would the server prevent duplicate XP under concurrency?

The service should authenticate the user, validate the occurrence and create a completion or claim inside a database transaction. A unique constraint would be the final protection against concurrent duplicate requests. XP could be derived from reward records or updated atomically only when an insert succeeds. A retry should return the existing result rather than grant another reward.

Follow-up / evidence: Client-side button disabling alone cannot enforce server-side uniqueness.

40. How would you handle offline synchronization?

I would persist pending client operations with stable IDs and send them when a connection returns. The server would make retries idempotent and attach versions or timestamps to accepted changes. Conflicts need an explicit policy, especially for completion versus undo and schedule edits. I would keep the server authoritative for shared rewards while showing a clear pending-sync state in the app.

Follow-up / evidence: Do not casually promise conflict-free sync or exactly-once network delivery.

Ownership, tradeoffs and interview delivery

41. What would you improve first?

I would replace large preference-backed collections with a structured transactional repository, move disk work off the UI thread and separate business logic from the activity more consistently. Next I would improve immutable day/schedule history, clock handling, diagnostics and restore atomicity. I would prioritize correctness and maintainability before adding multiplayer or cosmetic features.

Follow-up / evidence: Choose one improvement and explain the specific current limitation it addresses.

42. Did you use AI tools to build the project?

Yes. I used AI assistance to generate and iterate on implementation, tests, documentation and packaging. I should be clear about that assistance and explain which code paths and design decisions I have personally reviewed and understood. In an interview I would demonstrate understanding by tracing a feature, discussing its tradeoffs and making a small change, rather than claiming I wrote every line without assistance.

Follow-up / evidence: Adapt this answer truthfully to your actual review and learning. Do not invent independent work.

43. Why should a backend team value this project?

It gives concrete examples of state modeling, idempotency, time-based rules, validation, serialization and edge-case testing in Java. Those concepts transfer to backend services even though the runtime here is Android. I would combine this project with my separate Spring Boot/API projects to demonstrate both domain-logic reasoning and server-side implementation experience.

Follow-up / evidence: Keep evidence from different projects separate; do not attach Spring Boot/MySQL claims to Rise.

44. How would you demonstrate it in three minutes?

First, show a quest, its selected-day alarm and completion/undo behavior. Next, explain the dated completion key and show the XP/rank rules. Then preview a preset and start a timed event, explaining the stored deadline. Finish by opening one test for an edge case and describing the packaging bug. Keep the demo focused on implemented logic, not only the appearance of the screens.

Follow-up / evidence: Practice follow-ups: What breaks at scale? What happens at midnight? How would a server enforce this rule?